AWS Certified Machine Learning Engineer Associate — MLA-C01

SECTION 3 • PART 1 of 2

Data Transformation, Integrity & Feature Engineering

EMR • Apache Spark • Missing Data • Outliers • Unbalanced Data • Feature Engineering Techniques

EMR & Spark	Missing Data	Unbalanced Data	Outlier Handling	Feature Engineering

Section 3 — Division Plan

Section 3 is divided into two parts to ensure deep understanding of each concept. Here is the complete breakdown of what each part covers:

PART	Topics Covered	Focus Area
PART 1 (This PDF)	1. EMR (Elastic MapReduce) & Hadoop 2. Apache Spark — Architecture & MLlib 3. Handling Missing Data — All Techniques 4. Handling Unbalanced Data — SMOTE & more 5. Handling Outliers — Standard Deviation, RCF 6. Feature Engineering Techniques • Binning, Transforming, Encoding • Scaling/Normalization, Shuffling • TF-IDF & Curse of Dimensionality	**Big Data Processing + Core Data Science Foundations**
PART 2 (Next PDF)	1. SageMaker Data Processing Deep Dive 2. SageMaker Ground Truth & Labeling 3. SageMaker Data Wrangler 4. SageMaker Feature Store 5. SageMaker Model Monitor & Clarify 6. AWS Glue — ETL & Data Quality 7. AWS Glue DataBrew 8. Amazon Athena — Advanced Features	**AWS Managed ML Services + Data Quality Tools**

Table of Contents — Part 1

1. EMR (Elastic MapReduce) — Overview & Architecture
2. EMR Cluster — Node Types & Usage Patterns
3. EMR Storage — HDFS, EMRFS, EBS
4. EMR Serverless & EMR on EKS
5. EMR Instance Type Selection Guide
6. Apache Hadoop & Spark — Architecture Deep Dive
7. Spark Components — MLlib, Streaming, SQL
8. Handling Missing Data — All Techniques
9. Handling Unbalanced Data — SMOTE & Strategies
10. Handling Outliers — Standard Deviation & RCF
11. Feature Engineering — What & Why
12. Curse of Dimensionality
13. Binning & Quantile Binning
14. Data Transformations — Log, Power, etc.
15. Encoding — One-Hot, Label, Target Encoding
16. Scaling & Normalization
17. Shuffling Training Data
18. TF-IDF — Text Feature Engineering
19. Part 1 Cheat Sheet & Exam Tips

1. Amazon EMR — Elastic MapReduce

Amazon EMR is AWS's fully managed big data platform built on **Apache Hadoop**. It allows you to process and analyze massive datasets using popular frameworks like Spark, Hive, Presto, Flink, and HBase on a cluster of EC2 instances. EMR is the primary service for large-scale data transformation and ML feature engineering on AWS.

1. What is EMR?

- **Full name:** Elastic MapReduce — named after the MapReduce programming model
- **Foundation:** Managed Hadoop framework running on EC2 instances
- **Frameworks included:** Apache Spark, HBase, Presto, Flink, Hive, Pig, and more
- **EMR Notebooks:** Jupyter-based notebooks backed by an EMR cluster for interactive analysis
- **Pricing:** Charged per hour for EC2 instances used + EMR overhead

1.2 Why Use EMR for ML?

- **Scale:** Process petabytes of raw data that won't fit in memory of a single machine
- **Speed:** Spark on EMR processes data in-memory — 100x faster than traditional MapReduce
- **Integration:** Native AWS integration with S3, Glue, SageMaker, Kinesis, CloudWatch
- **ML Frameworks:** Run Spark MLlib, TensorFlow, scikit-learn directly on EMR clusters
- **Cost-efficiency:** Use Spot instances for task nodes — save up to 90% on compute costs

1.3 EMR AWS Integrations

AWS Service	How EMR Uses It	ML Relevance
Amazon EC2	Instances that form the cluster nodes	Compute for ML processing
Amazon S3	Input/output data, model artifacts, logs	Primary data lake storage
Amazon VPC	Network isolation for the cluster	Security for sensitive ML data
Amazon CloudWatch	Monitor cluster performance, set alarms	Track training job metrics
AWS IAM	Control permissions for cluster operations	Secure data access
AWS Glue	Data catalog integration for schema discovery	Auto-discover ML datasets
AWS CloudTrail	Audit all API calls to EMR	Compliance logging
AWS Data Pipeline	Schedule and start EMR clusters on schedule	Automated ML pipelines

2. EMR Cluster — Node Types & Architecture

An EMR cluster consists of three types of nodes, each with a distinct role. Understanding this architecture is critical for both the exam and real-world usage.

1. The Three Node Types

① Master Node

- **Role:** The brain of the cluster — manages and coordinates all other nodes
- **Count:** Always exactly 1 per cluster (single EC2 instance)
- **Responsibilities:** Tracks job status, schedules tasks, monitors node health
- **Recommended instance:** m4.large for clusters <50 nodes; m4.xlarge for >50 nodes
- **Critical:** If master node fails, the entire cluster fails — do NOT use Spot instances here

② Core Node

- **Role:** Stores data AND runs computational tasks
- **Storage:** Hosts HDFS (Hadoop Distributed File System) data
- **Count:** At least 1 required. Can be scaled up/down (with caution)
- **Risk:** Removing core nodes risks data loss from HDFS — scale carefully
- **Recommended instance:** m4.large for most workloads
- **Instance choice:** Use On-Demand or Reserved for stability

③ Task Node

- **Role:** ONLY runs tasks — does NOT store HDFS data
- **Purpose:** Add extra compute capacity without risk of data loss
- **Scaling:** Safe to add and remove at any time — ideal for auto-scaling
- **Cost trick:** Best candidate for Spot instances — if terminated, no data loss
- **ML use case:** Spin up many task nodes during model training, remove when done

Node Type	Runs Tasks?	Stores HDFS?	Count	Spot OK?	Typical Instance
Master	Yes (schedules)	No	Exactly 1	■ Never	m4.large / m4.xlarge
Core	Yes	Yes	≥1	■■ Risky	m4.large
Task	Yes	No	0 or more	■ Ideal	Any compute type

■■ **Exam Alert:** Master node = single point of failure. NEVER use Spot instances for master or core nodes in production. Task nodes are the ONLY safe candidate for Spot instances.

2.2 EMR Usage Patterns

- **Transient Clusters:** Spin up → process data → shut down. Pay only for what you use. Best for batch ML feature engineering jobs
- **Long-Running Clusters:** Persistent cluster for ongoing work. Use Reserved Instances to save costs. Connect directly to master node to run jobs interactively
- **Spot Instances for Task Nodes:** Dramatically cut costs for temporary processing capacity
- **EMR Serverless:** AWS manages nodes automatically — no manual sizing needed
- **Submit Steps:** Submit ordered processing steps via the console or CLI

3. EMR Storage Options

EMR supports multiple storage systems. Choosing the right one significantly affects performance, cost, and durability of your ML data processing pipelines.

1. HDFS — Hadoop Distributed File System

- **What it is:** The native distributed file system that splits files across all core nodes
- **Speed:** Fastest storage for EMR — data is local to the compute nodes
- **Durability:** Data is replicated across nodes (default 3x replication)
- **Limitation:** Data is LOST when the cluster terminates (ephemeral)
- **Best for:** Intermediate processing data, shuffle data during Spark jobs
- **Backed by:** EBS volumes attached to core nodes

3.2 EMRFS — EMR File System (S3-backed)

- **What it is:** Allows EMR to access Amazon S3 AS IF it were HDFS
- **Syntax:** Use s3:// paths instead of hdfs:// paths in your code
- **Persistence:** Data survives cluster termination — stored safely in S3
- **Best for:** Input data, final output, model artifacts — anything that must persist
- **Cost:** S3 pricing (much cheaper than EBS for storage)
- **EMRFS Consistent View:** Optional feature that uses DynamoDB to track consistency of S3 operations (prevents race conditions when multiple jobs write to same S3 path)

3.3 Local File System & EBS

- **Local file system:** Each node has local disk storage for temp files, Spark shuffle
- **EBS for HDFS:** Attach EBS volumes to increase HDFS storage capacity on core nodes

Storage	Backed By	Persists After Cluster?	Speed	Best For
HDFS	EBS on Core nodes	■ No	Fastest	Intermediate/shuffle data
EMRFS	Amazon S3	■ Yes	Fast	Input, output, ML artifacts
Local FS	Instance storage	■ No	Fastest	Temp files, OS operations
EBS (for HDFS)	EBS volumes	■ No	Fast	Expand HDFS storage

■ **Tip:** Best practice for ML pipelines: Store your raw input data and final output in S3 (via EMRFS) for durability. Use HDFS for intermediate processing steps within the job for maximum speed.

4. EMR Serverless & EMR on EKS

1. EMR Serverless

EMR Serverless eliminates the need to configure and manage cluster nodes. You simply choose an EMR release and runtime, submit your job, and AWS automatically provisions the right resources.

- **How it works:** Submit jobs via API/CLI. EMR computes needed resources and schedules workers
- **Supported runtimes:** Spark, Hive, Presto
- **Multi-AZ:** Workers distributed across multiple AZs within a region
- **Pre-initialized capacity:** Optionally pre-warm workers to eliminate cold starts
- **Spark memory overhead:** Spark adds ~10% overhead — ensure initial capacity is 10% MORE than what job requests

EMR Serverless Application Lifecycle

You must manage the lifecycle manually via API calls — it does NOT happen automatically:

Step	API Call	Description
CreateApplication	Create the serverless application	(Spark, Hive, etc.)
StartApplication	Start the application (workers bec	ome available)
StartJobRun	Submit your Spark scripts or Hive	queries
StopApplication	Stop when done to avoid charges	
DeleteApplication	■■ Must delete to fully stop billi	ng!

■■ **Exam Alert:** Forgetting to call DeleteApplication will continue to incur charges even if no jobs are running! This is a common exam gotcha.

4.2 EMR on EKS

- **Purpose:** Submit Spark jobs directly to Elastic Kubernetes Service (EKS) without provisioning EMR clusters
- **Benefit:** Share Kubernetes resources between Spark and other applications
- **Fully managed:** No cluster management needed
- **Use case:** Organizations already running EKS that want to add Spark workloads

5. EMR Instance Type Selection Guide

Choosing the right EC2 instance type for EMR significantly impacts performance and cost. The exam tests your ability to match workload requirements to instance families.

Workload Type	Recommended Instance	Why?
General ML / Default	m4.large / m4.xlarge	Balanced compute + memory — good all-rounder
Web crawling / External I/O	t2.medium	Lots of waiting on external resources — burstable
Compute-intensive (NLP, ML)	High CPU (c4, c5)	Many CPU cores for parallel processing
In-memory analytics / caching	High Memory (r4, r5)	Large memory for large datasets in RAM
Deep Learning / GPU tasks	GPU (g3, g4, p2, p3)	GPU acceleration for neural networks
HPC / Distributed training	Cluster Compute (p4d)	High bandwidth networking between nodes
Cost-saving task nodes	Spot Instances (any type)	Save 60-90% — safe since task nodes hold no data

■ **Tip:** EMR Exam Rule: Master = m4.large (or xlarge for big clusters). Core = m4.large. Task = Spot instances. GPU workloads = p2/p3/g3/g4. Never use Spot for master or core nodes.

6. Apache Hadoop & Spark — Architecture Deep Dive

1. Apache Hadoop — The Foundation

Hadoop is the open-source framework that revolutionized big data processing. EMR is built on top of Hadoop and provides managed access to its ecosystem.

- **HDFS (Hadoop Distributed File System):** Splits large files into 128MB blocks, distributes them across nodes, replicates each block 3x for fault tolerance
- **YARN (Yet Another Resource Negotiator):** Resource manager — allocates CPU and memory to jobs running on the cluster
- **MapReduce:** Original processing engine. Reads from disk → Maps → Shuffles → Reduces → Writes to disk. Very slow due to disk I/O between stages

6.2 Apache Spark — Why It Replaced MapReduce

Apache Spark is the modern in-memory processing engine that is 100x faster than MapReduce for most ML workloads. It runs ON TOP of Hadoop's YARN and HDFS but processes data in memory rather than writing to disk between stages.

- **In-memory processing:** Keeps intermediate data in RAM — eliminates disk I/O bottleneck
- **DAG execution:** Builds a Directed Acyclic Graph of operations, optimizes before executing
- **Lazy evaluation:** Transformations are not executed until an action is called
- **Fault tolerance:** Uses RDD lineage to recompute lost partitions
- **Multi-language:** Supports Python (PySpark), Scala, Java, R, SQL

6.3 How Spark Works — Architecture

Understanding Spark's architecture helps you troubleshoot performance issues and design efficient ML pipelines:

- **Driver Program:** The main program that creates the SparkContext and orchestrates the job
- **SparkContext:** The entry point to Spark — connects to the cluster manager
- **Cluster Manager:** Allocates resources (YARN on EMR, or Spark's own standalone manager)
- **Executors:** Worker processes on each node that run tasks and store data in cache
- **Tasks:** Individual units of work sent to executors — operate on data partitions

Component	Role	Location	ML Implication
Driver Program	Orchestrates job, holds SparkContext	Master node	Memory-intensive — use large instance
SparkContext	Connection to cluster, creates RDDs	Driver	Single entry point for all operations
Cluster Manager	Allocates CPU/RAM to executors	YARN/Master	YARN used on EMR clusters
Executors	Run tasks, cache data partitions in RAM	Worker nodes	More executors = more parallelism
Tasks	Smallest unit of work — operates on 1 partition	Executor	Tune partition count for efficiency

7. Spark Components — MLlib, Streaming, SQL

7.1 Spark Core Components

Spark is not just a processing engine — it's a unified analytics platform with specialized modules for different use cases:

Component	Purpose	ML/Data Relevance
Spark Core	Base engine — distributed task scheduling, memory management, fault tolerance	Foundation for all other components
Spark SQL	SQL interface + DataFrames API for structured data processing	Query structured ML feature tables
Spark Streaming	Real-time stream processing using micro-batches (DStreams)	Online feature engineering from streams
Spark MLlib	Distributed ML library — algorithms run across entire cluster	Train ML models at scale on EMR
GraphX	Graph computation framework	Graph-based ML, social network analysis

7.2 Spark MLlib — Distributed Machine Learning

Spark MLlib is a distributed ML library that runs algorithms across an entire EMR cluster. It is designed for when your dataset is too large for a single machine.

- **Classification:** Logistic Regression, Naïve Bayes, Decision Trees, Random Forests, GBTs
- **Regression:** Linear Regression, Generalized Linear Models, Gradient-Boosted Trees
- **Clustering:** K-Means, Gaussian Mixture, LDA (topic modeling), Bisecting K-Means
- **Recommendation:** Alternating Least Squares (ALS) — collaborative filtering
- **Dimensionality Reduction:** PCA, SVD (Singular Value Decomposition)
- **Feature Engineering:** TF-IDF, Word2Vec, StandardScaler, StringIndexer, OneHotEncoder
- **ML Pipelines:** Chain transformers and estimators into reproducible pipelines

7.3 Spark Structured Streaming

Spark Structured Streaming enables real-time stream processing using the same DataFrame API as batch processing — making it easy to build unified batch+stream ML pipelines.

```
val inputDF = spark.readStream.json("s3://logs")
inputDF.groupBy($"action", window($"time", "1 hour")).count()
.writeStream.format("jdbc").start("jdbc:mysql//...")
```

- **Kinesis integration:** Read from Kinesis Data Streams using Spark Dataset API with KCL
- **Use case:** Real-time feature computation, streaming anomaly detection, online model updates

7.4 Zeppelin & EMR Notebooks

- **Apache Zeppelin:** Interactive web-based notebook for Spark. Run Spark code interactively, visualize results, execute SQL — makes Spark feel like a data science tool
- **EMR Notebooks:** Jupyter-based notebooks backed by EMR cluster. Backed up to S3, accessible via AWS Console only, hosted inside a VPC
- **Use case:** Exploratory data analysis, iterative feature engineering, model prototyping

8. Handling Missing Data — All Techniques

Missing data is one of the most common real-world problems in ML. How you handle it significantly impacts model accuracy. The exam tests all these techniques and expects you to know which is best in which situation.

1. Technique 1: Mean/Median/Mode Replacement (Imputation)

Replace missing values with a statistical measure computed from the non-missing values in the same column.

- **Mean replacement:** Replace with the average of the column
  - Fast, easy, preserves sample size
  - Does NOT change the mean of the column
  - Problem: Reduces variance — underestimates variability in data
  - Problem: Ignores correlations between features
  - Problem: Only works for numerical data
- **Median replacement:** Use when outliers are present — more robust than mean
- **Mode replacement:** Use for categorical data — replace with the most frequent value

■■ **Exam Alert:** Mean replacement is described as 'generally pretty terrible' in AWS documentation. It only works at the column level, misses feature correlations, and cannot handle categorical features. Know its limitations!

8.2 Technique 2: Dropping Rows with Missing Data

- **When acceptable:** Not many rows have missing data AND dropping them does not bias the dataset
- **Quick & easy:** No imputation needed — just remove incomplete rows
- **Big limitation:** This is NEVER the 'best' approach — almost anything is better
- **Alternative:** Can you substitute another similar field? (e.g., review summary vs. full review text)

8.3 Technique 3: ML-Based Imputation (Best Methods)

Use machine learning algorithms to predict and fill in missing values. More accurate but more complex.

- **KNN (K-Nearest Neighbors) Imputation:**
  - Find K most similar rows (nearest neighbors) and average their values
  - Works well for numerical data
  - For categorical data, use Hamming distance — but deep learning is usually better for categoricals
- **Deep Learning Imputation:**
  - Build a neural network model to predict the missing feature from other features
  - Especially powerful for categorical data
  - Complex to implement but can be very accurate
- **Regression Imputation:**
  - Find linear or nonlinear relationships between the missing feature and other features
  - Train a regression model using rows where the feature IS present
  - Predict missing values using that model
- **MICE (Multiple Imputation by Chained Equations) — Most Advanced:**
  - Imputes each missing variable using separate regression models
  - Runs multiple passes (chains) until convergence
  - Creates multiple complete datasets, averages results
  - Gold standard for statistical imputation

8.4 Technique 4: Just Get More Data

- The best solution to missing data is often to collect more complete, real data
- Imputation always introduces some bias — real data is always better

Technique	Best When	Limitation	Exam Ranking
Mean/Median/Mode	Quick prototype, numeric data	Ignores correlations, reduces variance	Worst ■
Drop Rows	Very few missing rows, no bias	Loses data, never the 'best' answer	Bad ■
KNN	Moderate missing data, numeric	Slower on large datasets	Good ■
Deep Learning	Categorical data, complex patterns	Requires more code and compute	Great ■
Regression	Clear linear/nonlinear relationship	Assumes relationship between features	uGood ■
MICE	When accuracy is critical	Complex, slow	Best ■ ■
Get More Data	Always if possible!	Not always feasible	Best ■ ■

9. Handling Unbalanced Data

Unbalanced data occurs when one class (the 'positive' or minority class) has far fewer examples than the other class. This is extremely common in real-world ML problems and causes models to be biased toward the majority class.

1. Understanding the Problem

- **Classic example — Fraud Detection:** 99% of transactions are legitimate, 1% are fraud
- **What happens:** A model that always predicts 'not fraud' achieves 99% accuracy — but is useless!
- **Terminology:** 'Positive' = the rare class you care about (fraud, disease). 'Negative' = the common class (legitimate transaction, healthy)
- **Primary concern:** Neural networks are especially susceptible to this problem
- **Better metrics:** Use Precision, Recall, F1-Score, AUC-ROC — NOT accuracy — for unbalanced data

9.2 Technique 1: Oversampling the Minority Class

- **Idea:** Duplicate existing samples from the minority class to increase its representation
- **Simple random oversampling:** Randomly duplicate minority class records
- **Advantage:** No data is lost, simple to implement
- **Disadvantage:** Creates exact duplicates → model may overfit to those specific examples

9.3 Technique 2: Undersampling the Majority Class

- **Idea:** Randomly remove samples from the majority class to balance the dataset
- **Advantage:** Reduces training time (smaller dataset)
- **Disadvantage:** Throws away real data — you lose potentially useful information
- **When appropriate:** When you specifically need to avoid big data scaling issues

9.4 Technique 3: SMOTE — Synthetic Minority Over-sampling Technique (Best Method)

SMOTE is the gold standard for handling unbalanced data. It generates **synthetic** (artificial) new examples rather than just duplicating existing ones.

- **How SMOTE works step by step:**
  - 1. For each sample in the minority class, find its K nearest neighbors (also minority class)
  - 2. Randomly select one of those K neighbors
  - 3. Create a new synthetic sample as a random point BETWEEN the original sample and the selected neighbor
  - 4. This interpolation creates realistic-looking new data points
  - 5. Also applies undersampling to the majority class
- **Advantages over simple oversampling:**
  - Creates NEW examples (not duplicates) → reduces overfitting
  - Generates realistic data by interpolating between real examples
  - Combines synthetic minority creation WITH majority undersampling
- **AWS support:** SageMaker Data Wrangler has built-in SMOTE support

9.5 Technique 4: Adjusting Classification Threshold

- **Idea:** Lower the probability threshold at which a sample is classified as positive
- **Default threshold:** 0.5 — if P(fraud) > 0.5, classify as fraud
- **Lower threshold:** If P(fraud) > 0.3, classify as fraud → catches more fraud (higher recall)

- **Trade-off:** Lower threshold → more true positives but also more false positives
- **Exam note:** Guaranteed to reduce false negatives; may increase false positives

Technique	How It Works	Pros	Cons
Oversampling	Duplicate minority samples	Simple, no data loss	Can overfit to duplicates
Undersampling	Remove majority samples	Faster training	Loses real data
SMOTE ■	Generate synthetic minority samples	Realistic new data, less overfit	More complex
Threshold Adjust	Change classification cutoff	Easy, no retraining	Trade precision for recall

■■ **Exam Alert:** SMOTE is generally the best answer for handling unbalanced data on the exam. SageMaker Data Wrangler supports it natively. Remember it also does undersampling of the majority class — it's a two-step approach.

10. Handling Outliers

Outliers are data points that are significantly different from other observations. They can distort model training and lead to poor generalization. However, removing them is not always appropriate — context matters enormously.

1. Variance and Standard Deviation — The Math

Standard deviation provides a principled, mathematical basis for identifying outliers.

- **Variance (σ^2):** The average of the squared differences from the mean
  - Formula: $\sigma^2 = \Sigma(x■ - \mu)^2 / N$
  - Example: For dataset (1, 4, 5, 4, 8): Mean = 4.4, Variance = 5.04
- **Standard Deviation (σ):** Square root of variance — in the same units as the data
  - $\sigma = \sqrt{(\text{variance})} \rightarrow \sigma = \sqrt{5.04} \approx 2.24$ for the example above
  - Tells you how far a typical point deviates from the mean
- **Using σ to detect outliers:**
  - Points more than 1σ from mean = unusual
  - Points more than 2σ from mean = significantly unusual
  - Points more than 3σ from mean = very likely an outlier (captures 99.7% of normal data)
  - 'How many sigmas away' = how extreme the outlier is

2. When to Remove Outliers

Removing outliers is a significant decision. Do it responsibly and with a clear reason.

- **Good reasons to remove:**
  - Collaborative filtering: A single user who rates 10,000 movies may skew everyone's recommendations
  - Web log data: Outliers often represent bots/scrapers that should be excluded
  - Sensor data: Outliers may represent measurement errors (e.g., a temperature reading of 500°C)
- **Bad reasons to remove:**
  - Just to make your model look better — this is data manipulation
  - Income data: Don't remove billionaires just because they're extreme — they ARE the data
  - Medical data: Extreme values may represent the rare cases you most need to predict

3. AWS Random Cut Forest (RCF) — Automated Outlier Detection

AWS developed the Random Cut Forest algorithm specifically for automated outlier (anomaly) detection. It appears across multiple AWS services and is a key exam topic.

- **How RCF works:** Builds a forest of random trees where each tree partitions the training data. Measures the expected change in complexity when a new point is added. Points that cause a large change = outliers (anomalous)
- **Assigns an anomaly score** to each data point — higher score = more anomalous
- **Unsupervised:** No labels needed — learns what 'normal' looks like from the data
- **Handles time series:** Detects spikes, breaks in periodicity, pattern changes
- **Where RCF appears in AWS (very commonly tested!):**
  - Amazon QuickSight — anomaly detection in business dashboards
  - Kinesis Data Analytics — RANDOM_CUT_FOREST SQL function for streaming data
  - Amazon SageMaker — built-in RCF algorithm for training custom models
  - Amazon Lookout for Metrics — uses RCF internally

■ Key Concept: Random Cut Forest (RCF) = AWS's proprietary anomaly detection algorithm. It appears in QuickSight, Kinesis Analytics, and SageMaker. If the exam asks about automated anomaly/outlier detection → RCF!

11. Feature Engineering — What & Why

Feature engineering is the process of using domain knowledge and data understanding to create, transform, and select input features that improve model performance. Andrew Ng famously said: *'Applied machine learning is basically feature engineering.'*

1. What is Feature Engineering?

- **Definition:** Applying knowledge of the data AND the model to create better features for training
- **Key questions it answers:**
  - Which features should I use?
  - Do I need to transform these features in some way?
  - How do I handle missing data in features?
  - Should I create NEW features from existing ones?
  - Which features are most predictive for this specific problem?

2. Why Can't You Just Use Raw Data?

- Raw data is often poorly scaled, correlated, noisy, or in wrong formats for ML algorithms
- Different algorithms have different requirements — linear models need normalized features; tree models are more forgiving
- Domain knowledge can create features that capture the real signal better than raw measurements
- **Example:** Instead of using 'birth_date', engineer 'age' and 'age_group' features

3. Feature Engineering in AWS

Task	AWS Tool	Notes
Visual feature transformation	SageMaker Data Wrangler	300+ built-in transforms, no code needed
Distributed feature engineering	EMR + Spark (MLlib)	For petabyte-scale datasets
Feature storage & sharing	SageMaker Feature Store	Online + offline store for ML features
ETL-based transforms	AWS Glue	Serverless Spark for feature prep
Notebook-based engineering	SageMaker Notebooks	Pandas, scikit-learn, NumPy available

12. The Curse of Dimensionality

The curse of dimensionality refers to the exponential increase in data sparsity as the number of features (dimensions) grows. It is one of the key reasons feature selection is crucial in ML.

- **The problem:** Every feature you add creates a new dimension in the data space
- **Sparsity:** As dimensions increase, data points become increasingly sparse — there are not enough training samples to cover the space
- **Distance metrics fail:** In high dimensions, all points become roughly equidistant from each other — K-NN and clustering algorithms break down
- **Overfitting risk:** Too many features with too few samples → model memorizes training data, fails on new data

1. Solutions to Dimensionality

- **Feature Selection:** Choose only the most relevant features — discard the rest
  - Domain knowledge: Ask 'does this feature logically predict the target?'
  - Correlation analysis: Remove features highly correlated with each other
  - Feature importance: Use tree models to rank feature importance
- **Dimensionality Reduction (Unsupervised):** Compress many features into fewer dimensions
  - **PCA (Principal Component Analysis):** Projects data onto directions of maximum variance. Available in SageMaker as a built-in algorithm and in Spark MLlib
  - **K-Means:** Can be used to cluster and represent data in fewer dimensions
  - **Autoencoders:** Neural networks that learn compressed representations

■■ **Exam Alert:** High dimensionality is often where domain expertise matters most. The exam may ask: 'Dataset has 500 features but only 100 samples' → Answer = dimensionality reduction (PCA) and/or feature selection.

13. Binning & Quantile Binning

Binning (also called discretization) converts continuous numerical features into discrete categorical buckets. This can make models more robust to outliers and noise in the data.

1. Simple (Equal-Width) Binning

- **Idea:** Divide the value range into N equal-width intervals (bins)
- **Example — Age:** 0-10, 10-20, 20-30, 30-40, 40-50... (each bin is 10 years wide)
- **Problem:** Uneven bin populations — the 20-30 bin may have 1000 people, the 90-100 bin may have 5

2. Quantile Binning (Better Approach)

- **Idea:** Create bins such that each bin contains EQUAL numbers of data points
- **Example:** If you have 1000 people and create 10 bins, each bin has exactly 100 people
- **Advantage:** Ensures even representation — no bin is over- or under-represented
- **Result:** Transforms numerical data into ORDINAL data (ordered categories)

3. When to Use Binning

- When there is **uncertainty in measurements** (e.g., estimated age from behavior)
- When you want to make the model **less sensitive to exact values**
- When the **relationship between feature and target is not smooth**
- When dealing with **sparse data** at the extremes of a distribution
- **Example use case:** BMI values → Underweight / Normal / Overweight / Obese buckets

■ **Note:** Binning converts numerical → categorical/ordinal. This changes what algorithms work best. Tree-based models handle ordinal data naturally; linear models need one-hot encoding of bins.

14. Data Transformations

Transformations apply mathematical functions to features to make them better suited for model training. The right transformation depends on the distribution of the data and the type of model being trained.

1. Log Transformation

- **Purpose:** Compress the range of right-skewed data (data with a long right tail)
- **Formula:** x_new = log(x)
- **When to use:** Feature data shows exponential growth pattern
- **Example:** Income data, website traffic, word frequencies (for TF-IDF)
- **YouTube recommendation system:** Uses log transform on video watch time features

2. Power Transformations

- **Square root:** x_new = $\sqrt{x}$ — moderate compression of right-skewed data
- **Square:** x_new = x^2 — amplifies larger values, useful for polynomial features
- **Box-Cox:** Family of power transformations that finds the optimal λ parameter

3. Polynomial Features

- **Idea:** Create new features by raising existing ones to powers: x, x^2, $\sqrt{x}$
- **Purpose:** Allows linear models to learn super-linear AND sub-linear relationships
- **Example (YouTube):** Feature x also represented as x^2 and $\sqrt{x}$ in the same model — lets the model learn any smooth function of x
- **AWS reference:** Google/YouTube deep learning paper on recommendation systems uses this technique

Transform	Formula	Fixes	Use Case
Log	log(x)	Right-skewed / exponential data	Income, traffic, frequencies
Square Root	$\sqrt{x}$	Moderate right skew	Count data, Poisson distributed
Square	x^2	Amplify large values	Polynomial regression features
Box-Cox	Variable λ	Various skewness types	When optimal transform unknown
Reciprocal	1/x	Left-skewed / inverse relationships	Speed from time, rates

15. Encoding Categorical Features

Most ML algorithms require numerical input. Encoding converts categorical features (text, labels, categories) into numbers that the model can process. Choosing the right encoding method matters significantly for model performance.

1. One-Hot Encoding (Most Common)

- **Idea:** Create a separate binary column for each unique category
- **Example:** Color = ['Red', 'Green', 'Blue'] → Three columns: Is_Red, Is_Green, Is_Blue
- **Each row:** Has a 1 in the column matching its category, 0 in all others
- **Best for:** Nominal categories (no ordering) with a small number of unique values
- **Deep Learning:** Very common — output neurons in neural networks often use one-hot targets
- **Problem:** High cardinality categories (e.g., zip codes with 10,000 values) → too many columns!

2. Label (Ordinal) Encoding

- **Idea:** Assign each category a unique integer: Red=0, Green=1, Blue=2
- **Best for:** Ordinal categories where order matters (Small=0, Medium=1, Large=2)
- **Problem:** For nominal data, implies ordering that doesn't exist (Blue > Red is meaningless)
- **Safe to use with:** Tree-based models (Decision Trees, Random Forest, XGBoost) that don't interpret numeric values as ordered for nominal categories

3. Target (Mean) Encoding

- **Idea:** Replace each category with the mean of the target variable for that category
- **Example:** City = 'Paris', and average house price in Paris = $500K → encode Paris as 500000
- **Advantage:** Handles high cardinality features well, captures target relationship
- **Risk:** Data leakage if not done carefully — must use cross-validation when computing means

Encoding	How It Works	Best For	Avoid When
One-Hot	Binary column per category	Nominal, low cardinality	High cardinality (>50 unique)
Label	Integer per category	Ordinal OR tree-based models	Linear models with nominal data
Target	Mean target per category	High cardinality nominal	Small datasets (overfit risk)
Hashing	Hash function → fixed-size vector	Very high cardinality	When interpretability needed
Embedding	Learn dense representation	NLP, recommendation systems	Small simple datasets

■■ **Exam Alert:** One-hot encoding is the most commonly tested encoding on the exam. Deep learning output layers use it for multi-class classification. For high cardinality features, use target encoding or embeddings.

16. Scaling & Normalization

Scaling brings all features into a comparable range. This is crucial because many ML algorithms (especially neural networks and distance-based algorithms) are sensitive to the magnitude of feature values.

1. Why Scaling Matters

- **Problem without scaling:** Feature 'Income' ranges from 0–200,000. Feature 'Age' ranges from 0–100. The model treats Income as 2,000x more important than Age just because of scale!
- **Neural networks:** Require features normalized around 0 — gradient descent converges much faster
- **Distance-based models (KNN, SVM, K-Means):** Dominated by large-scale features without scaling
- **Tree models (Decision Trees, Random Forest, XGBoost):** NOT sensitive to feature scale — no need to scale

1. Scaling Methods

- **Min-Max Scaling (Normalization):** Scales all values to [0, 1]
  - Formula: x_scaled = (x - min) / (max - min)
  - Use when: You need values in a bounded range, data has no extreme outliers
  - Problem: Sensitive to outliers — one extreme value compresses all others
- **Standardization (Z-score Normalization):** Centers data around 0 with unit variance
  - Formula: x_scaled = (x - mean) / std_deviation
  - Result: Mean = 0, Standard Deviation = 1
  - Use when: Data is normally distributed, algorithm assumes Gaussian features
  - More robust to outliers than Min-Max
- **Robust Scaling:** Uses median and IQR (interquartile range) instead of mean/std
  - Best when: Data has significant outliers that you cannot remove

2. AWS Implementation

- **scikit-learn:** MinMaxScaler, StandardScaler, RobustScaler — available in SageMaker notebooks
- **Spark MLlib:** MinMaxScaler, StandardScaler — for distributed large-scale feature scaling
- **SageMaker Linear Learner:** Has automatic normalization built in as a preprocessing step
- **CRITICAL:** Save your scaler parameters (min, max, mean, std) from training data. Apply SAME transformation to inference data!

■■ **Exam Alert:** You MUST scale training and inference data with the SAME parameters. If you fit a MinMaxScaler on training data, transform inference data using the SAME min/max values — do NOT refit on inference data. Otherwise your predictions will be wrong!

17. Shuffling Training Data

Shuffling randomizes the order of training examples before feeding them to the model. This is a simple but important preprocessing step.

- **Why shuffling matters:** If data is collected in order (e.g., first all healthy patients, then all sick patients), the model may learn from residual ordering signals rather than actual features
- **Mini-batch training:** Shuffling ensures each mini-batch contains a representative mix of classes and patterns — better gradient estimates
- **Epoch-level shuffling:** Re-shuffle at the start of each epoch — prevents the model from memorizing the order of examples
- **Prevents learning spurious patterns:** Without shuffling, the model may overfit to whatever ordering happens to exist in the raw data

■ **Note:** In SageMaker, the Linear Learner algorithm recommends shuffling the input data. Spark MLlib's MLPipeline supports shuffling as part of the data preparation stage.

18. TF-IDF — Text Feature Engineering

TF-IDF (Term Frequency - Inverse Document Frequency) is one of the most important techniques for converting raw text into numerical features for ML models. It captures how important a word is to a specific document within a collection.

1. Breaking Down TF-IDF

- **Term Frequency (TF):** How often a word appears in a SPECIFIC document
  - High TF = word is important to this particular document
  - Example: 'python' appears 15 times in a programming article → high TF for that doc
- **Document Frequency (DF):** How often a word appears across ALL documents
  - High DF = common word that appears everywhere (like 'the', 'a', 'is')
  - These common words tell us nothing about what makes a document unique
- **Inverse Document Frequency (IDF):** log(N / DF) where N = total number of documents
  - Words appearing in FEW documents get a HIGH IDF score — they're distinctive!
  - Common words appearing in MOST documents get a LOW IDF score
  - Log is used because word frequencies follow a power law distribution
- **TF-IDF Score = TF × IDF**
  - High score = word appears often in THIS document but rarely across all documents = highly relevant!
  - Low score = word is either rare in this doc OR common everywhere = not distinctive

2. Practical Example

Consider these two documents in a corpus of 1,000 articles:

- Word 'the' → TF: high, DF: very high (in 990/1000 docs) → IDF: very low → TF-IDF: LOW
- Word 'transformer' → TF: medium, DF: low (in 5/1000 docs) → IDF: very high → TF-IDF: HIGH
- Conclusion: 'transformer' is much more useful as a feature for document classification!

3. N-Grams — Extending TF-IDF

- **Unigrams:** Individual words — 'machine', 'learning', 'model'
- **Bigrams:** Pairs of consecutive words — 'machine learning', 'learning model'
- **Trigrams:** Triplets — 'deep learning model'
- **Why N-grams?** 'not good' means opposite of 'good' — unigrams miss this context
- **Trade-off:** Higher N = more context captured BUT exponentially more features → dimensionality

4. TF-IDF on Spark/EMR

- **Spark MLlib:** HashingTF + IDF pipeline for distributed TF-IDF computation
- **Words as hash values:** Convert words to integers for computational efficiency
- **Bag of Words assumption:** TF-IDF treats documents as unordered word collections — word order doesn't matter
- **Preprocessing needed:** Tokenization, lowercase, remove stop words, stemming/lemmatization
- **Search application:** Compute TF-IDF for all documents, sort by score for a query word

Concept	Formula	Meaning	Value When High
TF	count(word in doc) / total words	Word frequency in THIS doc	Word repeated often in doc
IDF	log(total docs / docs containing word)	Word rarity across ALL docs	Word appears in few documents

Concept	Formula	Meaning	Value When High
TF-IDF	TF × IDF	Word relevance to this document	Distinctive, important word

19. Part 1 Cheat Sheet & Exam Tips

1. EMR Quick Reference

- **Master node:** 1 per cluster, m4.large/xlarge, NEVER use Spot, manages cluster
- **Core node:** Stores HDFS + runs tasks, use On-Demand, risky to remove
- **Task node:** Compute only, IDEAL for Spot instances, safe to add/remove
- **EMRFS:** S3 as HDFS, data persists after cluster shutdown
- **EMR Serverless:** Must call DeleteApplication or charges continue!

2. Missing Data Decision Tree

Situation	Best Approach
Accuracy is critical, complex data	MICE (Multiple Imputation by Chained Equations)
Categorical missing data, complex patterns	Deep Learning imputation
Numerical data, moderate complexity	KNN imputation
Linear relationships between features	Regression imputation
Very quick prototype / exploration	Mean/Median replacement (numeric)
Very few missing rows, no bias risk	Drop rows (but never 'best' answer)
Feasible to collect more data	Get more data — always best option

19.3 Unbalanced Data Decision Tree

Situation	Best Approach
Best overall method, realistic new samples	SMOTE (Synthetic Minority Over-sampling)
Need to reduce false positives specifically	Increase classification threshold
Need to reduce false negatives specifically	Decrease classification threshold
Quick & dirty, small dataset	Simple oversampling (duplicate minority)
Need to reduce training time	Undersampling (but loses real data)

19.4 Feature Engineering Quick Reference

Technique	Input → Output	When to Use
Binning	Continuous → Ordinal	Uncertain measurements, reduce sensitivity to exact values
Log Transform	Skewed → Normal-ish	Right-skewed data, exponential growth patterns
One-Hot Encoding	Nominal → Binary columns	Categorical with few unique values (<50)
Label Encoding	Ordinal → Integer	Ordinal data OR tree models with nominal data
Min-Max Scaling	Any range → [0,1]	Neural networks, when bounded range needed

Technique	Input → Output	When to Use
Standardization	Any → mean=0, std=1	When algorithm assumes Gaussian features
Shuffling	Ordered → Random order	Always — before training any iterative model
TF-IDF	Text → Numerical matrix	Text classification, search, document similarity
PCA	N features → K features	High dimensionality, curse of dimensionality

■ Top 10 Exam Tips for Section 3 Part 1

1	**EMR Task Nodes = Spot Instances** Task nodes hold NO HDFS data, so it's always safe to use Spot instances for them. NEVER use Spot for master or core nodes in production.
2	**SMOTE = Best for Unbalanced Data** SMOTE generates SYNTHETIC new minority samples + undersamples majority. Data Wrangler supports it natively. It's almost always the best answer.
3	**RCF Appears Everywhere** Random Cut Forest = AWS anomaly detection algorithm. It appears in QuickSight, Kinesis Analytics (RANDOM_CUT_FOREST function), and SageMaker.
4	**MICE = Best Missing Data Imputation** Multiple Imputation by Chained Equations is the most advanced and accurate imputation technique. Know it by name for the exam.
5	**Parquet/ORC on EMR = Cost Savings** Columnar formats (Parquet, ORC) reduce Athena query costs 30-90%. Use CREATE TABLE AS SELECT in Athena to convert formats.
6	**EMRFS = Persistent, HDFS = Ephemeral** Data in HDFS is LOST when the cluster terminates. Use S3 (via EMRFS) for anything that must survive cluster shutdown.
7	**One-Hot = Nominal, Label = Ordinal** Use one-hot for categories with no order (colors, cities). Use label encoding for ordered categories (S/M/L) or tree models.
8	**Always Scale Before Neural Networks** Neural networks require normalized features (ideally mean=0, std=1). Tree models (XGBoost, Random Forest) do NOT need scaling.
9	**Mean Imputation = Generally Terrible** The AWS docs call mean replacement 'generally pretty terrible'. It ignores correlations and only works at the column level. Know its limitations.
10	**EMR Serverless Billing Trap** You MUST call DeleteApplication to stop all billing. Just stopping jobs is NOT enough — the application itself still incurs charges.

Part 1 Complete! ■ → Next: Part 2 covers SageMaker Data Tools, Glue, Athena & Data Quality